

FGLBA: Enabling Highly-Effective and Stealthy Backdoor Attack on Federated Graph Learning

Qing Lu[†], Miao Hu^{†*}, Di Wu[†], Yipeng Zhou[‡], Mohsen Guizani[§], and Quan Z. Sheng[‡]

[†] Sun Yat-sen University, Guangzhou, Guangdong, China

[‡]Macquarie University, Macquarie Park, NSW 2113, Australia

[§]Mohamed Bin Zayed University of Artificial Intelligence, Abu Dhabi, UAE

luqing8@mail2.sysu.edu.cn, {humiao5, wudi27}@mail.sysu.edu.cn,

{yipeng.zhou, michael.sheng}@mq.edu.au, mguizani@ieee.org

Abstract—Federated graph learning (FGL) has risen as a promising paradigm for collaboratively training graph neural networks while safeguarding data privacy. Nevertheless, the distributed nature of FGL also renders it susceptible to backdoor attacks. Although backdoor attacks are recognized as a significant threat to both centralized graph learning and federated learning (FL), the study of such attacks in FGL remains very limited. Current research on FGL backdoor attacks often merely adapts centralized graph backdoor attacks or FL backdoor attacks designed for image classification tasks to the FGL context, leaving key issues such as the effectiveness of triggers and the stealthiness of malicious models largely unexplored. To bridge this research gap, in this paper, we propose a novel backdoor attack, named FGLBA, targeting the FGL paradigm. Specifically, we design an input-aware trigger generator that generates a customized trigger for each target node based on its feature vector and neighborhood information, making that poisoned nodes injected with triggers are more likely misclassified into the category specified by the attacker. Additionally, we develop a stealthy federated backdoor training strategy that leverages collaborative optimization among multiple malicious clients to circumvent existing server-side defenses. The trigger generator and malicious clients' local models are iteratively optimized through a bilevel optimization framework, enabling the malicious models to achieve optimal attack performance under the optimal trigger generator. Extensive experiments on 4 real-world datasets demonstrate the effectiveness and superiority of our attack, outperforming all baseline attacks and successfully bypass 6 state-of-the-art and classical FL backdoor defenses.

Index Terms—backdoor attack, federated graph learning

I. INTRODUCTION

Recently, federated graph learning (FGL), which allows many graph data owners to collaboratively train GNNs without directly exposing their local graph data [1], has seen broad applications in plenty of security-sensitive domains like medical data mining [2] and fraud detection [3].

However, the distributed nature of FGL also introduces vulnerabilities that can be exploited by backdoor attackers. In such attacks, attackers aim to plant a “backdoor” into a model

during the training phase, causing it to respond in an attacker-specified way when encountering inputs that contain certain patterns, known as “triggers”, while functioning normally otherwise [4]. In FGL, backdoor attackers can corrupt one or more clients to submit malicious local models to the server, thereby contaminating the global model. Once the contaminated global model is deployed, attackers can activate the backdoor via trigger-embedded inputs. For example, in fraud detection tasks, attackers can lead a backdoor model to misclassify a fraudulent node as benign by simply adding some interactions between this node and some specific nodes (i.e., attaching a trigger to this node), thereby posing a significant threat.

Yet, backdoor attacks targeting FGL are largely unexplored. Current efforts either directly apply backdoor attacks in traditional federated learning (FL) that performs image classification or natural language processing tasks to the graph learning scenario, or simply federalize centralized graph backdoor attacks. Neither of these approaches takes into account the distinctive characteristics of the FGL setting, thereby yielding unsatisfactory attack results.

The design of the backdoor attacks on FGL is non-trivial, facing at least three challenges: 1) Existing FGL backdoor attacks [5] directly employ a randomly generated small graph as the trigger, neglecting the impact of the trigger's design on the attack effectiveness, resulting in a low attack success rate. How to design effective triggers for FGL remains an open issue. 2) Unlike graph learning in the centralized setting, FL has its unique defenses against backdoor attacks, thereby posing different requirements on the stealthiness of backdoor attacks. Directly federalizing centralized graph backdoor attacks will be largely mitigated by server-side defenses. How to enhance the stealthiness of backdoor attacks on FGL to circumvent existing backdoor defenses in FL is an unresolved issue. 3) In contrast to centralized scenarios with a single attacking entity, the federated setting allows attackers to corrupt several malicious clients to amplify the attack's impact, negating the dilution of attack effects during model aggregation. However, if each malicious client attacks independently without coordination, it would render them more susceptible to detection by server-side defenses. How these corrupted malicious clients coordinate their attack strategies to jointly ensure the effectiveness and stealthiness of the attack needs further investigation.

This work was supported by Shenzhen Science and Technology Program (Grant No. KJZD20230923113901004), the Science and Technology Planning Project of Guangdong Province under Grant 2023A0505020006, the Science and Technology Development Fund, Macau SAR (0008/2022/AGJ), the National Natural Science Foundation of China under Grant U2001209 and Grant 62072486, and the Fundamental Research Funds for the Central Universities, Sun Yat-sen University under Grant 23ptpy115, 24lqgb020.

* Corresponding author

To address the aforementioned challenges, in this paper, we propose a Federated Graph Learning Backdoor Attack (FGLBA). To tackle challenge 1), we design an input-aware trigger generator that leverages GNN's message-passing mechanism to craft customized triggers. To address challenges 2) and 3), we devise a stealthy federated backdoor training approach, enabling multiple malicious clients to collaboratively optimize their attack effectiveness in a stealthy manner. During FGL training, the trigger generator and malicious clients' local models are iteratively optimized in a bilevel optimization framework, ensuring that the malicious models achieve optimal attack performance under the optimal trigger generator. Our main contributions are as follows:

- We propose FGLBA, a novel backdoor attack targeting FGL, which employs a bilevel optimization framework between the trigger generator specially designed for graph-structured data and the local models of malicious clients to optimize the attack efficacy and stealthiness.
- We develop an input-aware backdoor trigger generation model with an encoder-decoder architecture to generate customized triggers for each target node by fully leveraging its features and neighborhood information, boosting the effectiveness of graph backdoor attacks.
- We design a stealthy federated backdoor training method that employs collaborative optimization among multiple malicious clients to circumvent the potential server-side defenses against backdoor attacks in the FGL system.
- Through extensive experiments on 4 public graph datasets with varying scales, we demonstrate the superiority of FGLBA. FGLBA achieves an attack success rate of over 95% even under the state-of-the-art backdoor defenses in FL, outperforming all baseline attacks.

II. RELATED WORK

In centralized graph learning, Zhang *et al.* [6] propose a backdoor attack for GNN-based graph classification tasks, which utilizes the Erdős-Rényi (ER) model [7] to create a random universal trigger for all input graphs. Xi *et al.* [8] propose a graph backdoor attack called GTA, which generates triggers tailored to individual graphs by optimizing the trigger generation function and mixing function. Dai *et al.* [9] present an unnoticeable graph backdoor attack (UGBA) that introduces an unnoticeable loss for the trigger generator, and reduces the attack budget via a clustering-based node selection algorithm.

Recently, a few studies attempt to explore the backdoor attacks on FGL. Xu *et al.* [5] investigate the application of the centralized backdoor attack (CBA) and the distributed backdoor attack (DBA) [10] in traditional FL to FGL. The universal trigger of CBA and DBA is also randomly generated by the ER model [7]. These attacks are designed for graph classification tasks, which differ from our scenario of node classifications. Furthermore, their experimental results show that even when there is no defense, the attack success rates of both DBA and CBA are far from satisfactory, indicating significant room for enhancement. Liu *et al.* [11] propose a benchmark called Bkd-FedGNN to evaluate backdoor attacks on federated GNN

through extensive experiments. However, the attacks evaluated in Bkd-FedGNN simply involve a straightforward federated implementation of centralized graph backdoor attacks, without considering the unique backdoor defenses in FL, as well as the potential for collusion among malicious clients.

III. PRELIMINARIES

A. Federated Graph Learning

Let $G = (\mathcal{V}, \mathcal{E}, \mathcal{X})$ denote an attributed graph, where $\mathcal{V} = \{v_i\}_{i=1}^N$ is the set of N nodes, $\mathcal{E} \subseteq \mathcal{V} \times \mathcal{V}$ is the set of edges, and $\mathcal{X} = \{\mathbf{x}_i\}_{i=1}^N$ is the set of node features with $\mathbf{x}_i$ representing the feature vector of node v_i . Let $\mathbf{A} \in \mathbb{R}^{N \times N}$ denote the adjacency matrix of G , where $\mathbf{A}_{ij} = 1$ if v_i and v_j are connected; otherwise $\mathbf{A}_{ij} = 0$. In this paper, we focus on the transductive node classification tasks, which are prevalent in real-world applications. Given a graph G and a set of labeled nodes $\mathcal{V}_L \subseteq \mathcal{V}$, the goal of transductive node classification is to infer the classes of the remaining unlabeled nodes $\mathcal{V}_U = \mathcal{V} \setminus \mathcal{V}_L$. Let $\mathcal{D} := (G, \mathcal{Y})$ denote the graph dataset, where $\mathcal{Y} = \{y_i | v_i \in \mathcal{V}_L\}$ is the set of labels with y_i representing the label of node v_i .

We consider an FGL system with n clients and one server. Let $\mathcal{N}$ denote client set. Each client $s \in \mathcal{N}$ holds a graph dataset $\mathcal{D}^{(s)} = (G^{(s)}, \mathcal{Y}^{(s)})$ locally. All clients collaboratively train a global model F parameterized by θ by solving

$$\min_{\theta \in \mathbb{R}} \sum_{s=1}^n \frac{N_{s,L}}{N_L} \mathcal{L}(\mathcal{D}^{(s)}, \theta), \quad (1)$$

where $N_{s,L} = |\mathcal{V}_L^{(s)}|$ and $N_L = \sum_{s \in \mathcal{N}} |\mathcal{V}_L^{(s)}|$ are the number of labeled nodes in client s 's local graph and the number of labeled nodes across all clients' graphs, respectively. $\mathcal{L}$ is the local objective function for each client defined as

$$\mathcal{L}(\mathcal{D}^{(s)}, \theta) = \frac{1}{N_{s,L}} \sum_{i=1}^{N_{s,L}} l(F(G^{(s)}(v_i), \theta), y_i^{(s)}), \quad (2)$$

where $G^{(s)}(v_i)$ denotes the computation graph of node v_i in graph $G^{(s)}$ and $l(\cdot)$ is the cross-entropy loss.

B. Threat Model

Attacker's goal: Our FGLBA attack aims to cause the global model to misclassify the nodes attached with triggers into the pre-determined target class y' , while leaving the accuracy on clean nodes uninfluenced.

Attacker's knowledge and capability: The attacker can corrupt m out of the total n clients. Let $\mathcal{M}$ denote the set of malicious clients. Malicious clients can collude with each other. We assume that all clients participate in every round of FGL. The aggregation rules and defense mechanisms, as well as benign local models, are all agnostic to malicious clients.

IV. ATTACK FORMULATION

In this section, we dive into the challenges of FGL backdoor attacks and formally define the problem we aim to address.

Optimizing the trigger generator: The first step of a backdoor attack is data poisoning. Every malicious client s first randomly samples $\rho |\mathcal{V}_L^{(s)}|$ nodes from $\mathcal{V}_L^{(s)}$ as the target

nodes $\mathcal{V}_P^{(s)}$, where $\rho \in [0, 1]$ is the poisoning rate. In graph backdoor attacks, a trigger is conventionally characterized as a minor graph. Let g_i denote the trigger attached to node v_i and $|g_i|$ denote the number of nodes in g_i (i.e., trigger size), which is fixed to a predefined constant Δ . Then, every malicious client s poisons its dataset by attaching triggers to $\mathcal{V}_P^{(s)}$.

As mentioned in challenge 1) in Sec. I, previous FGL backdoor attacks [5] [11] predominantly employ random graphs as triggers. However, such random triggers may increase the difficulty for models to misclassify target nodes into the target class. For instance, in a social network, if a target node's true label is "basketball player" and the trigger nodes exhibit features that align closely with those typically followed by basketball players, this trigger may inadvertently amplify the target node's genuine attributes, making it harder for the model to misclassify this node as a "tennis player". Thus, to tackle challenge 1), malicious clients aim to learn a trigger generator H with parameters w , so that the aggregated model of malicious local models that trained on poisoned local graphs with triggers generated by H_w , classifies nodes with triggers into the target class while correctly categorizing other nodes. The optimization problem for H can be formulated as

$$\min_w J_H(\{\mathcal{D}^{(s)}\}_{s \in \mathcal{M}}, \{\theta_s\}_{s \in \mathcal{M}}, w), \quad (3)$$

where J_H is the objective function of the trigger generator:

$$J_H(\{\mathcal{D}^{(s)}\}_{s \in \mathcal{M}}, \{\theta_s\}_{s \in \mathcal{M}}, w) = \sum_{s \in \mathcal{M}} \frac{|\mathcal{V}_L^{(s)}|}{\sum_{s \in \mathcal{M}} |\mathcal{V}_L^{(s)}|} \mathcal{L}_C(Q(\mathcal{D}^{(s)}, w), \theta_{ma}), \quad (4)$$

where $\theta_{ma} = \frac{|\mathcal{V}_L^{(s)}|}{\sum_{s \in \mathcal{M}} |\mathcal{V}_L^{(s)}|} \sum_{s \in \mathcal{M}} \theta_s$ is the aggregated malicious model and $Q(\mathcal{D}, w)$ denotes the poisoned graph dataset obtained through injecting triggers generated by H_w into graph dataset $\mathcal{D}$. $\mathcal{L}_C$ is the classification loss defined as

$$\mathcal{L}_C(\mathcal{D}, \theta) = \frac{1}{|\mathcal{V}_L|} \left(\sum_{v_i \in \mathcal{V}_P} l(F(G(v_i), \theta), y') + \sum_{v_i \in \mathcal{V}_L \setminus \mathcal{V}_P} l(F(G(v_i), \theta), y_i) \right). \quad (5)$$

Enhancing the attack stealthiness: As highlighted in challenges 2) and 3) in Sec. I, simply transposing centralized graph backdoor attacks into FGL faces difficulties in evading FL backdoor defenses. Thus, to address these two challenges, malicious clients need to optimize their models to enhance attack stealthiness by collaboratively solving

$$\min_{\{\theta_s\}_{s \in \mathcal{M}}} J_F(\{\mathcal{D}^{(s)}\}_{s \in \mathcal{M}}, \{\theta_s\}_{s \in \mathcal{M}}, w), \quad (6)$$

where J_F is the objective function that considers both the effectiveness and the stealthiness of the attack, whose form is given in Sec. V-C in detail.

Bilevel optimization: To ensure that malicious models achieve the optimal attack performance under the optimized trigger generator, we take both (3) and (6) into consideration to formulate the problem as a bilevel optimization:

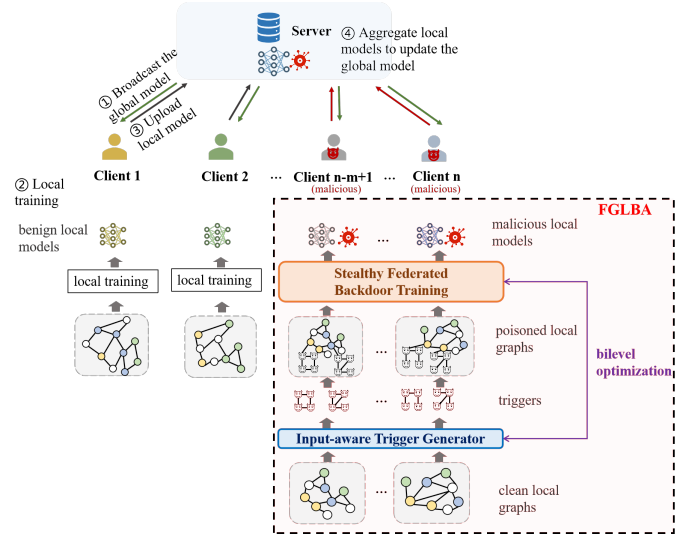


Fig. 1. The overall framework of FGLBA

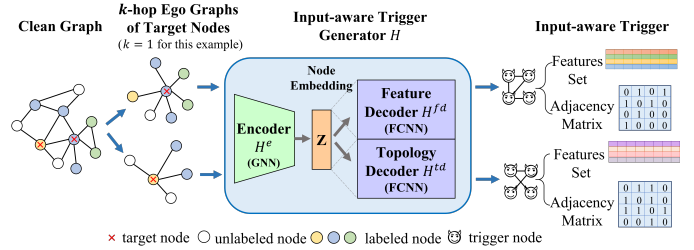


Fig. 2. Illustration of input-aware trigger generation

$$\min_{\{\theta_s\}_{s \in \mathcal{M}}} J_F(\{\mathcal{D}^{(s)}\}_{s \in \mathcal{M}}, \{\theta_s\}_{s \in \mathcal{M}}, w^*), \quad (7)$$

s.t. $w^* = \arg \min_w J_H(\{\mathcal{D}^{(s)}\}_{s \in \mathcal{M}}, \{\theta_s\}_{s \in \mathcal{M}}, w)$.

V. ATTACK METHODOLOGY

A. Attack Overview

Fig. 1 provides the overall framework of FGLBA. Unlike benign clients, malicious clients modify the local training process by training malicious local models on poisoned local graphs. FGLBA has two critical components: *input-aware trigger generator* and *stealthy federated backdoor training*.

B. Input-Aware Trigger Generator

Generally, GNNs derive a node's embedding by aggregating messages from its neighboring nodes and its own features. Therefore, node features and neighborhood information are crucial for the classification results. Given that each node has distinct features and neighbors, we develop an input-aware trigger generator to create a tailored trigger for each node.

To achieve this goal, we design a neural network with an encoder-decoder structure as the trigger generator. The encoder captures the feature vector and neighborhood information of a target node to generate the embedding of this node, while the decoder module maps this embedding to produce the trigger. The workflow of the input-aware trigger generation is shown in Fig. 2. Specifically, we employ a GNN as the encoder H^e , as GNNs are inherently designed to extract features from graph-structured data. Let $\phi_{i,k}$ denote the k -hop ego graph

of node v_i , which is defined as the graph comprising v_i and its k -step neighbors. H^e takes $\phi_{i,k}$ as input and outputs the node embedding $\mathbf{z}_i = H^e(\phi_{i,k}, \mathbf{w}_e)$, where $\mathbf{w}_e$ represents the learnable parameters of H^e . In our practice, $k = 1$ is sufficient to generate effective triggers as the core information for classification is often concentrated in the 1-hop neighborhood.

Since triggers contain both node features and topological information, we implement the decoder module with two fully connected neural networks (FCNNs): one as the feature decoder H^{fd} that generates the node features $\mathcal{X}_{trigger,i} = H^{fd}(\mathbf{z}_i, \mathbf{w}_{fd})$ and the other as the topology decoder H^{td} that generates the adjacency matrix $\mathbf{A}_{trigger,i} = H^{td}(\mathbf{z}_i, \mathbf{w}_{td})$, where $\mathbf{w}_{fd}$ and $\mathbf{w}_{td}$ are learnable parameters of H^{fd} and H^{td} , respectively. H^e , H^{fd} and H^{td} constitute the trigger generator H , and $\mathbf{w} = (\mathbf{w}_e, \mathbf{w}_{fd}, \mathbf{w}_{td})$.

Upon acquiring $\mathcal{X}_{trigger,i}$ and $\mathbf{A}_{trigger,i}$, the trigger $g_i = (\mathcal{V}_{trigger,i}, \mathcal{E}_{trigger,i}, \mathcal{X}_{trigger,i})$ is then uniquely determined. Then each malicious client s can obtain a poisoned local dataset $Q(\mathcal{D}^{(s)}, \mathbf{w}) = (G^{(s)} \odot \{g_i\}_{v_i \in \mathcal{V}_P^{(s)}}, \mathcal{V}^{(s)})$, where $G^{(s)} \odot \{g_i\}_{v_i \in \mathcal{V}_P^{(s)}}$ denotes the poisoned graph after connecting g_i to each target node $v_i \in \mathcal{V}_P^{(s)}$ on $G^{(s)}$. All components of the trigger generator H will be jointly optimized based on the objective function defined in (4) and (5) in Sec. IV.

C. Stealthy Federated Backdoor Training

In general, a backdoor attack is likely thwarted by server-side defenses if malicious models are overly similar to each other or significantly different from most models. Consequently, to maintain attack effectiveness under potential defenses, malicious clients should pursue three objectives:

Objective 1: Uploaded malicious models should misclassify nodes with triggers into the target class, while accurately classifying other nodes into their genuine classes.

Objective 2: Uploaded malicious models should not deviate too much from the “centroid” of the local models.

Objective 3: Uploaded malicious models should be as dissimilar as possible from each other.

To achieve Objective 1, each malicious client should train models on its local poisoned dataset with both clean and poisoned nodes, with an objective function formulated as

$$J_1(\{\mathcal{D}^{(s)}\}_{s \in \mathcal{M}}, \{\theta_s\}_{s \in \mathcal{M}}, \mathbf{w}) = \sum_{s \in \mathcal{M}} \frac{|\mathcal{V}_L^{(s)}|}{\sum_{s \in \mathcal{M}} |\mathcal{V}_L^{(s)}|} \mathcal{L}_C(Q(\mathcal{D}^{(s)}, \mathbf{w}), \theta_s), \quad (8)$$

where $\mathcal{L}_C$ is the classification loss defined in (5).

To fulfill Objective 2, a viable approach is to add a regularization term to malicious clients’ training objective to constrain the discrepancy between malicious models and the “centroid” of local models. Following [12], we consider the common scenario where the number of benign clients exceeds that of malicious ones. Hence, Objective 2 can be approximated by minimizing the discrepancy between malicious and benign models, which is equivalent to maximizing their similarity. Lacking access to benign clients’ local data, malicious clients need to estimate benign local models. Specifically, each malicious client first trains a non-toxic model $\theta_{b,s}$ on its clean

local dataset. Next, all malicious clients communicate to derive robust aggregated models θ_c , θ_g and θ_a from their non-toxic models, where θ_c , θ_g and θ_a represent the coordinate-wise median, geometric median and average aggregation of $\{\theta_{b,s}\}_{s \in \mathcal{M}}$, respectively, which can furnish malicious clients with more information about the “centroid” of benign models.

Then, malicious clients need to measure the similarity between the reference models $\Theta_{ref} = \{\theta_c, \theta_g, \theta_a, \{\theta_{b,s}\}_{s \in \mathcal{M}}\}$ and their models. Since existing FL backdoor defenses [12]–[15] primarily rely on Euclidean distance or cosine similarity, we adopt these two metrics to assess model similarity as

$$\text{sim}(\theta_i, \theta_j) = \beta_1 \cdot \frac{\theta_i \cdot \theta_j}{|\theta_i| \cdot |\theta_j|} + \beta_2 \cdot \|\theta_i - \theta_j\|_2, \quad (9)$$

where $\beta_1, \beta_2 \geq 0$ are the factors balancing the two metrics, and $\beta_1 + \beta_2 = 1$. More metrics can be incorporated into our similarity assessment by adding weighted terms in (9). The objective function for Objective 2 can be written as

$$J_2(\{\theta_s\}_{s \in \mathcal{M}}) = - \sum_{s \in \mathcal{M}} \sum_{\theta_{ref} \in \Theta_{ref}} \text{sim}(\theta_s, \theta_{ref}). \quad (10)$$

To achieve Objective 3, malicious clients need to collaboratively negotiate their uploaded models. We can leverage (9) to measure the similarity between two malicious models. The objective function for Objective 3 can be formulated as

$$J_3(\{\theta_s\}_{s \in \mathcal{M}}) = \sum_{s \in \mathcal{M}} \sum_{s' \in \mathcal{M} \setminus \{s\}} \text{sim}(\theta_s, \theta_{s'}). \quad (11)$$

To achieve all three objectives concurrently, we integrate J_1 , J_2 and J_3 by taking a weighted sum as

$$J_F(\{\mathcal{D}^{(s)}\}_{s \in \mathcal{S}}, \{\theta_s\}_{s \in \mathcal{M}}, \mathbf{w}) = J_1(\{\mathcal{D}^{(s)}\}_{s \in \mathcal{S}}, \{\theta_s\}_{s \in \mathcal{M}}, \mathbf{w}) + \lambda \cdot [(1 - \alpha) \cdot J_2(\{\theta_s\}_{s \in \mathcal{M}}) + \alpha \cdot J_3(\{\theta_s\}_{s \in \mathcal{M}})], \quad (12)$$

where $\lambda > 0$ is a factor that balances the effectiveness and stealthiness of the attack, while $\alpha > 0$ adjusts the contribution of the two regularization terms for different types of defenses.

In practice, α can be adaptively adjusted. Specifically, malicious clients can first calculate the minimum pair-wise similarity among reference models $\zeta_{ref} = \min\{\frac{\theta_s \cdot \theta_{s'}}{|\theta_s| \cdot |\theta_{s'}|} | \theta_s, \theta_{s'} \in \Theta_{ref}\}$ and the maximum pair-wise similarity among malicious models $\zeta_{max} = \max\{\frac{\theta_s \cdot \theta_{s'}}{|\theta_s| \cdot |\theta_{s'}|} | s, s' \in \mathcal{M}\}$. If a substantial gap exists between ζ_{ref} and ζ_{max} , α should be set higher to direct model updates to concentrate more on Objective 3. On the contrary, if ζ_{max} is near or below ζ_{ref} , α should be set lower to shift focus to Objective 2. We constrain the value of α within the range of 0 to 1 as $\alpha = \frac{1}{1 + e^{-(\zeta_{max} - \zeta_{ref})/\tau}}$, where $\tau > 0$ is a constant controlling the rate of change of α .

D. Optimization Algorithm

Exactly solving (7) is expensive. To address this, we use an approximate solution which iteratively refines $\{\theta_s\}_{s \in \mathcal{M}}$ and $\mathbf{w}$ through an alternating optimization process on J_F and J_H using gradient descent [16].

Updating the trigger generator: At the t -th iteration of local training, we first apply gradient descent to update $\mathbf{w}$ for U inner iterations with $\{\theta_s\}_{s \in \mathcal{M}}$ fixed to $\{\theta_s^{(t-1)}\}_{s \in \mathcal{M}}$:

$$\mathbf{w}^{(u)} = \mathbf{w}^{(u-1)} - \gamma_H \nabla_{\mathbf{w}} J_H(\{\mathcal{D}^{(s)}\}_{s \in \mathcal{M}}, \{\boldsymbol{\theta}_s^{t-1}\}_{s \in \mathcal{M}}, \mathbf{w}), \quad (13)$$

where γ_H is the learning rate for training the trigger generator.

Updating malicious models: In the outer iteration, we update $\{\boldsymbol{\theta}_s\}_{s \in \mathcal{M}}$ with $\mathbf{w}$ fixed to the updated $\mathbf{w}^{(U)}$ as

$$\boldsymbol{\theta}_s^{(t)} = \boldsymbol{\theta}_s^{(t-1)} - \gamma_F \nabla_{\boldsymbol{\theta}_s} J_F(\{\mathcal{D}^{(s)}\}_{s \in \mathcal{M}}, \{\boldsymbol{\theta}_s\}_{s \in \mathcal{M}}, \mathbf{w}^{(U)}), \quad (14)$$

where γ_F is the learning rate for training models.

FGLBA attack algorithm is summarized in Algorithm 1.

Algorithm 1 Federated Graph Learning Backdoor Attack

Input: The global model's parameter $\boldsymbol{\theta}_{glob}^{r-1}$ at the $(r-1)$ -th FGL round, the trigger generator's parameter $\mathbf{w}^{r-1}$ at the $(r-1)$ -th FGL round, the number of inner iterations U , the number of outer iterations T

Output: The parameters $\{\boldsymbol{\theta}_s^r\}_{s \in \mathcal{M}}$ of the uploaded malicious local models and the trigger generator's parameter $\mathbf{w}^r$ at the r -th FGL round

```

1:  $\boldsymbol{\theta}_s^{(0)} \leftarrow \boldsymbol{\theta}_{glob}^{r-1}$  for each  $s \in \mathcal{M}$ 
2:  $\mathbf{w}^{(0)} \leftarrow \mathbf{w}^{r-1}$ 
3: for  $t = 1, 2, \dots, T$  do
4:   for  $u = 1, 2, \dots, U$  do
5:     Update  $\mathbf{w}^{(u)}$  by (13)  $\triangleright$  Optimization of
       the Input-aware Trigger Generator
6:   end for
7:   Poison the local dataset  $\mathcal{D}^{(s)}$  with the updated trigger
       generator parameterized by  $\mathbf{w}^{(U)}$  for each  $s \in \mathcal{M}$ 
8:   Update  $\{\boldsymbol{\theta}_s^{(t)}\}_{s \in \mathcal{M}}$  by (14)  $\triangleright$  Stealthy
       Federated Backdoor Training
9:    $\mathbf{w}^{(0)} \leftarrow \mathbf{w}^{(U)}$ 
10: end for
11:  $\mathbf{w}^r \leftarrow \mathbf{w}^{(U)}$ 
12:  $\{\boldsymbol{\theta}_s^r\}_{s \in \mathcal{M}} \leftarrow \{\boldsymbol{\theta}_s^{(T)}\}_{s \in \mathcal{M}}$ 

```

VI. EXPERIMENTS

A. Experimental Setups

1) *Datasets:* Our experiments use four datasets of varying scales: small-scale citation networks CiteSeer [17] and PubMed [18], medium-scale graph Amazon-Computers [19], and large-scale citation network ogbn-arxiv [20]. We simulate a real-world FGL system with non-independent and non-identically distributed training data by partitioning a large graph into n subgraphs using the Louvain [21] algorithm, serving as clients' local graphs [22]. Each local graph is split into 60% labeled nodes, 15% test nodes (2/3 clean, 1/3 poisoned), and the rest for validation, with $\rho = 1/3$.

2) *Models:* We employ three GNN models in our experiments: GraphSage [23], GCN [24] and GAT [25]. Each model has an output layer with the same number of units as the input dataset, and a softmax layer followed by a logarithm is added after the last convolutional layer for classification. Specifically, we use GraphSage for CiteSeer and ogbn-arxiv, GCN for Amazon-Computers, and GAT for PubMed. The trigger generator employs a GAT with a single convolutional layer as the encoder, and two single-layer FCNNs as decoders.

3) *Baseline Attacks:* As mentioned in Sec. II, the CBA and DBA attacks proposed in [5] are designed for graph classification tasks, which cannot be applied to our scenario due to differences in trigger generation and attachment. Lacking FGL-specific backdoor attacks for node classification, we follow [11] to directly adapt state-of-the-art graph backdoor attacks in the centralized setting into FGL as baselines. Fed_SBA is a simple federated version of the attack from [6]. Fed_GTA and Fed_UGBA are the direct adaptations of GTA [8] and UGBA [9], respectively. Since GTA and UGBA are designed for a single adversary, we implement m malicious clients submitting identical malicious model updates.

4) *Implementation Details:* Our FGL system has 10 clients, with 3 being malicious ($n = 10, m = 3$). We set $\Delta = 3, \lambda = 2, \tau = 1$ and $k = 1$ for all datasets. β_1 is set to 0.995 for PubMed and 0.99 for others, resulting in $\beta_2 = 0.005$ for PubMed and 0.01 for others. The number of FGL communication rounds is 100 for CiteSeer and PubMed, and 200 for Amazon-Computers and ogbn-arxiv. Other hyperparameters are tuned to their optimal values in each experiment.

5) *Evaluation Metrics:* We evaluate backdoor attack performance using attack success rate (ASR) and clean accuracy (CA) [9]. ASR measures the global model's accuracy on poisoned test nodes with triggers, while CA measures its accuracy on clean test nodes. We conduct five tests for all experiments and take the average value as the final result.

B. Experimental Results

We compare FGLBA with baselines on 4 datasets, with results shown in Table I. Bold values indicate the highest ASR for each task. Only CA is reported in no-attack scenarios.

Attack performance without defenses: As shown in Table I, FGLBA achieves an ASR of over 98% on 4 datasets when there is no defense, outperforming all baselines. Its superiority is especially notable on complex tasks like ogbn-arxiv and Amazon-Computers. Additionally, FGLBA's CA drops by less than 3 percentage points compared to no attack, indicating strong classification performance on clean nodes.

Attack performance under various defenses: To assess the stealthiness of FGLBA, we compare it with baselines under 6 classical and state-of-the-art FL backdoor defenses. These defenses fall into two categories. The first category consists of detection-based methods, such as Foolsgold [14], Flame [13] and Multi-metrics [15], that identify and remove malicious clients by analyzing uploaded local models. The second category, such as Krum [12], CM [26] and RFA [27], devises robust aggregation rules to make the aggregated model resilient to malicious updates even if they are not excluded.

As shown in Table I, FGLBA achieves the highest ASR across all datasets and defenses, notably outperforming baselines under Foolsgold [14] and Flame [13]. This indicates superior stealthiness and effectiveness of FGLBA. Furthermore, under all evaluated defenses, FGLBA's CA is comparable to or even higher than that obtained without any attack, suggesting that FGLBA's performance on the main classification task is similar to or better than under no attack. This is because

TABLE I
ASR (%) AND CA (%) OF BACKDOOR ATTACKS UNDER DIFFERENT DEFENSES

Dataset	Attack	Defense													
		No defense		Foolsgold		Krum		Multi metrics		Median		RFA		Flame	
		ASR	CA	ASR	CA	ASR	CA	ASR	CA	ASR	CA	ASR	CA	ASR	CA
CiteSeer	No attack	-	75.03±0.80	-	75.20±0.93	-	59.64±1.33	-	74.90±0.60	-	74.90±0.74	-	75.10±0.92	-	54.12±2.42
	FGLBA	99.71±0.08	74.64±1.05	99.69±0.16	74.61±1.30	99.71±0.22	64.05±3.67	99.37±0.84	74.31±0.66	99.74±0.00	74.80±0.99	99.69±0.10	74.15±0.83	97.13±1.88	51.05±4.17
	Fed_SBA	95.82±0.63	75.16±1.21	95.98±0.66	74.84±1.21	22.43±38.63	56.99±6.68	88.25±6.83	73.53±1.10	95.64±0.60	74.51±1.02	95.04±0.31	75.10±0.94	14.83±28.48	47.25±13.95
	Fed_GTA	96.11±4.29	70.33±1.38	84.44±6.95	71.44±1.03	99.30±1.40	59.44±3.10	98.17±2.03	69.93±1.63	94.20±6.13	69.48±2.05	93.42±5.93	71.11±1.33	94.21±8.30	39.76±4.97
	Fed_UGBA	84.18±2.51	74.84±0.94	0.00±0.00	68.86±0.96	81.98±23.51	31.76±2.85	46.79±16.07	74.15±1.94	32.19±10.20	76.31±0.73	29.19±9.42	76.60±0.72	8.77±22.60	28.76±3.41
PubMed	No attack	-	85.31±0.15	-	39.95±0.00	-	84.24±0.45	-	84.65±0.44	-	85.40±0.18	-	85.36±0.14	-	84.35±0.38
	FGLBA	99.77±0.39	85.24±0.13	98.62±1.28	73.69±16.01	96.93±1.30	83.77±0.18	99.81±0.29	83.97±0.47	99.17±1.24	85.52±0.26	99.83±0.24	85.04±0.31	95.79±3.39	84.05±0.22
	Fed_SBA	70.41±4.39	84.87±0.15	68.51±5.49	84.73±0.37	1.40±0.52	83.85±0.22	57.87±9.23	84.16±0.19	46.73±5.15	84.83±0.24	55.23±5.65	84.80±0.24	1.48±0.58	84.51±0.30
	Fed_GTA	69.80±23.09	79.74±3.46	70.01±45.71	38.85±4.29	42.03±46.03	68.24±13.97	95.46±3.92	75.08±4.07	75.09±27.84	82.14±0.56	98.79±1.10	77.71±5.28	94.00±8.81	77.11±4.12
	Fed_UGBA	87.51±5.33	85.17±0.27	0.17±0.12	49.32±2.13	70.62±44.78	57.10±18.21	99.80±0.06	52.62±1.70	28.42±14.51	84.85±0.30	45.71±23.53	84.76±0.23	0.86±0.52	84.58±0.51
Amazon-Computers	No attack	-	81.02±1.06	-	15.60±0.00	-	50.87±15.24	-	66.09±2.07	-	64.03±1.90	-	69.87±1.18	-	65.32±0.41
	FGLBA	98.78±0.57	78.81±1.19	98.19±1.03	37.99±1.41	96.24±2.27	53.00±5.18	98.80±0.38	71.42±3.60	95.92±2.95	64.15±0.67	99.25±0.40	70.36±1.64	99.43±0.05	64.52±0.87
	Fed_SBA	61.29±3.91	81.39±3.39	0.00±0.00	15.90±0.89	90.83±4.22	53.60±5.18	77.71±5.94	69.07±4.25	19.88±13.50	62.95±2.08	68.96±3.71	78.32±4.26	80.19±3.44	65.13±0.51
	Fed_GTA	77.69±15.49	80.37±2.18	0.00±0.00	20.51±8.72	25.17±37.07	31.08±8.58	75.45±12.48	68.89±9.50	63.82±4.40	59.63±2.24	82.46±7.22	78.32±2.79	85.37±5.96	68.05±5.54
	Fed_UGBA	77.56±4.32	81.16±1.95	0.00±0.00	31.45±9.99	94.75±2.57	42.10±2.12	90.07±0.01	59.37±2.89	29.97±18.37	60.16±6.13	89.18±4.87	71.33±3.61	85.51±7.60	62.66±0.63
ogbn-arxiv	No attack	-	55.49±0.31	-	55.00±0.57	-	44.43±3.88	-	53.95±0.66	-	54.02±0.55	-	55.39±0.21	-	55.22±0.23
	FGLBA	99.08±0.37	55.28±0.49	99.34±0.33	55.31±0.31	98.71±0.23	48.12±2.60	98.05±1.06	54.46±0.18	98.54±1.72	53.93±0.90	99.27±0.16	55.23±0.43	98.63±1.21	54.97±0.35
	Fed_SBA	85.33±0.72	54.25±0.39	0.00±0.00	35.34±1.75	9.56±28.68	41.68±3.07	90.24±6.00	53.08±0.86	73.98±2.54	52.95±0.58	84.80±0.24	55.23±5.65	0.00±0.00	55.67±0.22
	Fed_GTA	83.23±20.86	47.84±1.42	0.00±0.00	14.81±1.01	33.73±22.47	20.62±1.27	45.09±28.25	45.22±2.08	57.28±40.81	42.94±2.93	91.37±15.76	45.79±3.14	67.89±33.27	44.68±4.71
	Fed_UGBA	95.43±0.43	53.54±0.34	0.00±0.00	13.80±0.07	97.21±1.06	22.17±5.29	98.01±0.29	50.71±0.63	95.31±0.67	50.60±0.67	97.25±0.31	52.33±0.68	19.72±39.43	55.01±1.44

defenses like Foolsgold may mistakenly exclude many benign models, leading to a loss of critical information for classifying clean nodes during aggregation. Under FGLBA, due to its stealthiness, malicious models can evade defenses and participate in final aggregation. While introducing a backdoor, they also positively contribute to the main classification task.

VII. CONCLUSION

In this paper, we propose FGLBA, a novel backdoor attack against FGL. We design an input-aware trigger generator that leverages both features and topological information of graph-structured data to customize triggers for each target node. Additionally, we develop a stealthy federated backdoor training method, where multiple malicious clients collaboratively optimize their models based on model similarity to evade existing defenses. Furthermore, we use bi-level optimization to ensure the optimality of the trigger generator and malicious local models, enhancing both the effectiveness and stealthiness of the attack. Extensive experiments validate FGLBA's outstanding attack performance. In future work, we plan to extend the FGLBA attack to graph classification and link prediction tasks, as well as explore effective defense mechanisms against the security threats of backdoor attacks in FGL.

REFERENCES

- [1] N. Cui, X. Wang, W. H. Wang, V. Chen, and Y. Ning, "Equipping federated graph neural networks with structure-aware group fairness," in *ICDM*, pp. 980–985, IEEE, 2023.
- [2] H. Chen, O. Engkvist, Y. Wang, M. Olivecrona, and T. Blaschke, "The rise of deep learning in drug discovery," *Drug Discovery Today*, vol. 23, no. 6, pp. 1241–1250, 2018.
- [3] B. Wang, J. Jia, and N. Z. Gong, "Graph-based security and privacy analytics via collective classification with joint weight learning and propagation," in *NDSS*, The Internet Society, 2019.
- [4] P. Chen, J. Yang, J. Lin, Z. Lu, Q. Duan, and H. Chai, "A practical clean-label backdoor attack with limited information in vertical federated learning," in *ICDM*, pp. 41–50, IEEE, 2023.
- [5] J. Xu, R. Wang, S. Koffas, K. Liang, and S. Picke, "More is better (mostly): On the backdoor attacks in federated graph neural networks," in *ACSAC*, p. 684–698, Association for Computing Machinery, 2022.
- [6] Z. Zhang, J. Jia, B. Wang, and N. Z. Gong, "Backdoor attacks to graph neural networks," in *SACMAT*, pp. 15–26, 2021.
- [7] E. N. Gilbert, "Random graphs," *The Annals of Mathematical Statistics*, vol. 30, no. 4, pp. 1141–1144, 1959.
- [8] Z. Xi, R. Pang, S. Ji, and T. Wang, "Graph backdoor," in *USENIX Security*, pp. 1523–1540, 2021.
- [9] E. Dai, M. Lin, X. Zhang, and S. Wang, "Unnoticeable backdoor attacks on graph neural networks," in *WWW*, pp. 2263–2273, 2023.
- [10] C. Xie, K. Huang, P.-Y. Chen, and B. Li, "Dba: Distributed backdoor attacks against federated learning," in *ICLR*, 2019.
- [11] F. Liu, S. Lai, Y. Ning, and H. Liu, "Bkd-FedGNN: A benchmark for classification backdoor attacks on federated graph neural network," *arXiv preprint arXiv:2306.10351*, 2023.
- [12] P. Blanchard, E. M. El Mhamdi, R. Guerraoui, and J. Stainer, "Machine learning with adversaries: Byzantine tolerant gradient descent," in *NeurIPS*, vol. 30, Curran Associates, Inc., 2017.
- [13] T. D. Nguyen, P. Rieger, R. De Viti, H. Chen, B. B. Brandenburg, H. Yalame, H. Möllering, H. Fereidooni, S. Marchal, M. Miettinen, et al., "FLAME: Taming backdoors in federated learning," in *USENIX Security*, pp. 1415–1432, 2022.
- [14] C. Fung, C. J. Yoon, and I. Beschastnikh, "The limitations of federated learning in sybil settings," in *RAID*, pp. 301–316, 2020.
- [15] S. Huang, Y. Li, C. Chen, L. Shi, and Y. Gao, "Multi-metrics adaptively identifies backdoors in federated learning," in *ICCV*, pp. 4652–4662, 2023.
- [16] D. Zügner and S. Günnemann, "Adversarial attacks on graph neural networks via meta learning," in *ICLR*, 2019.
- [17] C. L. Giles, K. D. Bollacker, and S. Lawrence, "Citeseer: an automatic citation indexing system," in *Proceedings of the Third ACM Conference on Digital Libraries*, p. 89–98, 1998.
- [18] P. Sen, G. Namata, M. Bilgic, L. Getoor, B. Galligher, and T. Eliassi-Rad, "Collective classification in network data," *AI Magazine*, vol. 29, no. 3, pp. 93–93, 2008.
- [19] O. Shchur, M. Mumme, A. Bojchevski, and S. Günnemann, "Pitfalls of graph neural network evaluation," *arXiv preprint arXiv:1811.05868*, 2018.
- [20] W. Hu, M. Fey, M. Zitnik, Y. Dong, H. Ren, B. Liu, M. Catasta, and J. Leskovec, "Open graph benchmark: Datasets for machine learning on graphs," *NeurIPS*, vol. 33, pp. 22118–22133, 2020.
- [21] V. D. Blondel, J.-L. Guillaume, R. Lambiotte, and E. Lefebvre, "Fast unfolding of communities in large networks," *Journal of Statistical Mechanics: Theory and Experiment*, vol. 2008, no. 10, p. P10008, 2008.
- [22] Z. Wang, W. Kuang, Y. Xie, L. Yao, Y. Li, B. Ding, and J. Zhou, "Federatedscope-gnn: Towards a unified, comprehensive and efficient package for federated graph learning," in *KDD*, pp. 4110–4120, 2022.
- [23] W. Hamilton, Z. Ying, and J. Leskovec, "Inductive representation learning on large graphs," *NeurIPS*, vol. 30, 2017.
- [24] T. N. Kipf and M. Welling, "Semi-supervised classification with graph convolutional networks," in *ICLR*, 2017.
- [25] P. Veličković, G. Cucurull, A. Casanova, A. Romero, P. Liò, and Y. Bengio, "Graph attention networks," in *ICLR*, 2018.
- [26] D. Yin, Y. Chen, R. Kannan, and P. Bartlett, "Byzantine-robust distributed learning: Towards optimal statistical rates," in *ICML*, pp. 5650–5659, PMLR, 2018.
- [27] K. Pillutla, S. M. Kakade, and Z. Harchaoui, "Robust aggregation for federated learning," *IEEE Transactions on Signal Processing*, vol. 70, pp. 1142–1154, 2022.